

SHETH MVLU COLLEGE

SUBJECT:- AI

Practical 6

AIM:- Support Vector Machines (SVM)

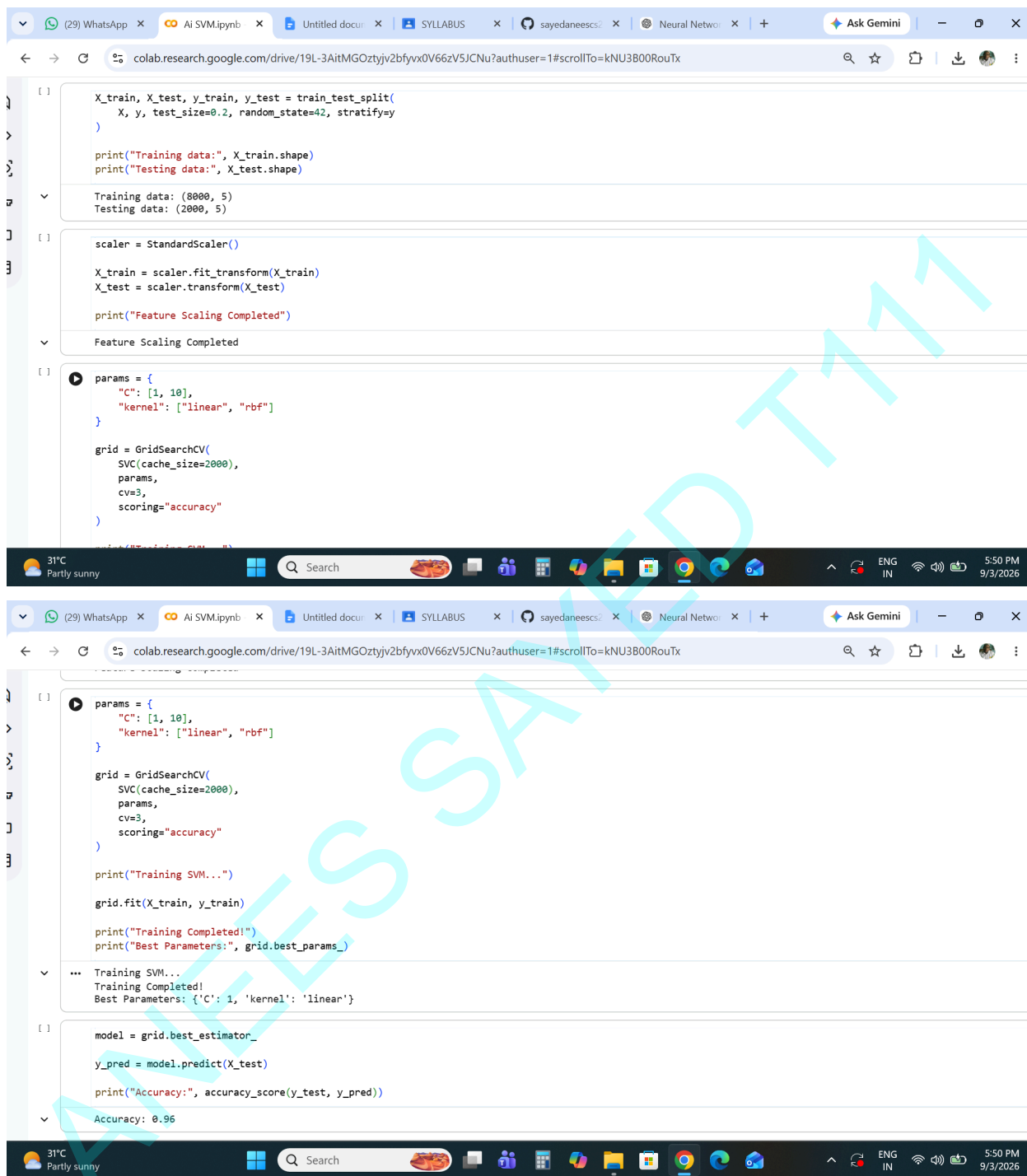
- Implement the SVM algorithm for binary classification.
- Train an SVM model using a given dataset and optimize its parameters.
- Evaluate the performance of the SVM model on test data and analyze the results.

The screenshot displays a Google Colab notebook interface with the following components:

- Browser Tabs:** (29) WhatsApp, Ai SVM.ipynb, Untitled docu..., SYLLABUS, sayedaneescs..., Neural Netwo..., Ask Gemini.
- Address Bar:** colab.research.google.com/drive/19L-3AitMGOZtyjv2bfyvx0V66zV5JCNu?authuser=1
- Notebook Menu:** File, Edit, View, Insert, Runtime, Tools, Help.
- Commands:** + Code, + Text, Run all.
- Code Cells:**
 - Cell 1: Imports pandas as pd, matplotlib.pyplot as plt, and various sklearn modules (train_test_split, GridSearchCV, StandardScaler, SVC, accuracy_score, confusion_matrix, classification_report, ConfusionMatrixDisplay).
 - Cell 2: Imports files from google.colab and uses files.upload() to upload a file named 'cybersecurity.csv'.
 - Cell 3: Loads the dataset using pd.read_csv("cybersecurity.csv"), prints the shape, and displays the first few rows.
- Output of Cell 3:** A table showing the first 5 rows of the 'cybersecurity.csv' dataset. The table has 13 columns: timestamp, src_ip, dst_ip, src_port, dst_port, protocol, bytes_sent, bytes_received, user_agent, url, is_internal_traffic, label, and attack_type.
- Code Cell 4:** Prepares the data for SVM by selecting features (src_port, dst_port, bytes_sent, bytes_received, is_internal_traffic) and the target variable (label).
- Code Cell 5:** Splits the data into training and testing sets using train_test_split.

SHETH MVLU COLLEGE

SUBJECT:- AI



The image displays two screenshots of a Google Colab notebook interface. The top screenshot shows the initial setup for training an SVM model. The bottom screenshot shows the completion of training and the resulting accuracy.

Top Screenshot Code:

```
[1] X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

Training data: (8000, 5)
Testing data: (2000, 5)

[2] scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Feature Scaling Completed")

Feature Scaling Completed

[3] params = {
    "C": [1, 10],
    "kernel": ["linear", "rbf"]
}

grid = GridSearchCV(
    SVC(cache_size=2000),
    params,
    cv=3,
    scoring="accuracy"
)
```

Top Screenshot Output:

```
Training data: (8000, 5)
Testing data: (2000, 5)

Feature Scaling Completed
```

Bottom Screenshot Code:

```
[4] params = {
    "C": [1, 10],
    "kernel": ["linear", "rbf"]
}

grid = GridSearchCV(
    SVC(cache_size=2000),
    params,
    cv=3,
    scoring="accuracy"
)

print("Training SVM...")

grid.fit(X_train, y_train)

print("Training Completed!")
print("Best Parameters:", grid.best_params_)

Training SVM...
Training Completed!
Best Parameters: {'C': 1, 'kernel': 'linear'}

[5] model = grid.best_estimator_

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.96
```

Bottom Screenshot Output:

```
Training SVM...
Training Completed!
Best Parameters: {'C': 1, 'kernel': 'linear'}

Accuracy: 0.96
```

SHETH MVLU COLLEGE

SUBJECT:- AI

